

A Confidence-Aware Pose-Sequence Pipeline for Video Fall Detection

Technical and Scientific Analysis of the Implementation

Publication-oriented technical report

Based exclusively on the accompanying Jupyter notebook implementation

Contents

1	Introduction	3
2	Project Overview	4
3	Technical Foundations	5
3.1	Binary temporal fall detection	5
3.2	Pose-based representation	5
3.3	Graph representation	5
3.4	Temporal convolution	6
3.5	Temporal self-attention	6
4	Data Sources and Input Processing	6
4.1	Dataset policy	6
4.2	Versioning and provenance	7
4.3	Secure acquisition and extraction	7
4.4	Manifest as the scientific boundary	7
5	Split Design and Leakage Control	7
6	Runtime and Resource Management	8
6.1	Three-hour target and hard ceiling	8
6.2	Pose throughput estimation	8
6.3	VRAM control	8
7	Pose Extraction Pipeline	9
7.1	Temporal sampling	9
7.2	Primary-person selection	9
7.3	Missing detections	9
7.4	Skeleton normalization	9
7.5	Velocity, acceleration, and speed	10
7.6	Pose cache integrity	10
8	Train-Only Standardization	11
9	Dataset Class and Augmentation	11
9.1	Tensor preparation	11
9.2	Skeleton-only augmentation	12
10	Model Architecture	12
10.1	Graph-convolutional spatial branch	12
10.2	Temporal branch	12
10.3	Gated fusion	13
10.4	Attention and classification	13
10.5	Baselines	13
11	Training Procedure	13
12	Validation and Threshold Calibration	14
12.1	Metric computation	14

12.2 Validation-only operating threshold	15
13 Model Suite and Selection Rule	15
14 Final Model Freezing	16
15 Held-Out Testing	16
15.1 GMDCSA24 subject-disjoint test	16
15.2 MCFD cross-view evaluation	16
15.3 CAUCAFall external evaluation	16
16 Targeted Failure Recovery	16
17 Hard-Negative Analysis	17
18 Reproducibility and Reliability	17
19 Implementation Deviations and Important Caveats	18
19.1 The confidence-aware claim is only partially realized	18
19.2 The spatial branch contains unused confidence code	19
19.3 Hard-negative threshold mismatch	19
19.4 Per-frame normalization	19
19.5 Temporal reversal augmentation	19
19.6 Heuristic person selection	19
19.7 View-shift versus independent-event generalization	19
20 Computational Complexity	20
21 Code Organization and Execution Semantics	20
22 Overall Technical Assessment	21
23 Conclusion	21

Abstract

This report provides a detailed technical analysis of a research-oriented fall-detection notebook that converts short video segments into compact human-pose sequences and classifies them as fall or non-fall. The implementation is deliberately designed for a constrained Google Colab/T4 environment. Its central strategy is to freeze a lightweight YOLO11n-Pose estimator, extract 17 COCO keypoints from 32 temporally sampled frames, normalize the skeletons, derive velocity, acceleration, and speed features, and train a small sequence classifier that combines graph-based spatial processing, temporal convolutions, gated fusion, and two-head temporal self-attention. Tiny MLP and GRU models are retained as baselines, while feature and structural ablations are treated as optional experiments.

The notebook also implements a substantial experimental-control layer. Dataset archives are resolved through versioned metadata, verified by checksums, cached on persistent storage, and extracted through a Zip Slip-safe routine. Dataset manifests preserve subject, camera, scenario, activity, label, and temporal interval information. Training and validation data are pose-cached before model learning; the held-out test sets are deliberately processed only after model selection. A cumulative runtime controller separates a three-hour target from a roughly four-hour hard ceiling, and explicit VRAM checks constrain peak allocated memory.

The implementation is technically thoughtful, especially in its attention to provenance, leakage control, cache integrity, resumable execution, validation-only threshold selection, and final artifact verification. At the same time, the source contains important mismatches between the documented intent and the executed behavior. Most notably, the default feature mask disables the confidence channel even though the proposed architecture computes confidence statistics, and the hard-negative analysis reconstructs binary predictions at threshold 0.5 rather than the final validation-calibrated threshold. These issues do not invalidate the overall pipeline, but they materially affect how the system should be described and interpreted. No performance claims are made in this report because the uploaded notebook does not contain completed model-training outputs that establish such claims.

1. Introduction

Falls are temporal events rather than purely static postures. A single image can show a person lying on the ground, but the same appearance can arise from benign activities such as lying down, sitting, kneeling, or other daily activities. A useful video-based fall detector therefore needs information about body configuration as well as its change over time.

The notebook analyzed here addresses that problem through a pose-based representation. Instead of training a large RGB video model, it separates perception from temporal classification. A frozen pose estimator converts each sampled video frame into body landmarks; the trainable model then operates on the resulting low-dimensional sequence. This separation is computationally attractive for an inexpensive GPU environment because most trainable computation is moved away from raw image tensors and toward structured pose features.

The notebook’s stated objective is not merely to produce a binary classifier. It attempts to establish a reproducible experimental workflow under a strict compute budget. The default configuration trains on MCFD and GMDCSA24, keeps CAUCAFall as an external dataset, uses YOLO11n-Pose as a frozen feature extractor, and evaluates a lightweight graph-temporal architecture against small baselines. The scientific design also explicitly refuses to label MCFD as a subject-independent benchmark because the dataset has very limited subject diversity and multiple synchronized camera views of the same underlying events.

This report reconstructs the implementation from the notebook itself. The analysis distinguishes between three levels whenever useful: the theoretical concept, the concrete implementation, and the practical consequence of that implementation choice. Notebook cell numbers are referenced

as traceability markers rather than as a substitute for explaining the underlying method.

2. Project Overview

At the highest level, the pipeline can be represented as the following sequence:

$$\begin{aligned}
 &\text{Video archive} \rightarrow \text{validated manifest} \rightarrow \text{temporal segment} \rightarrow 32 \text{ sampled frames} \\
 &\rightarrow \text{YOLO11n-Pose} \rightarrow 17\text{-joint skeleton} \rightarrow \text{normalized motion features} \\
 &\rightarrow \text{train-only standardization} \rightarrow \text{sequence model} \rightarrow \text{validation-calibrated rule.}
 \end{aligned} \tag{1}$$

The evaluation path intentionally branches from this common representation. Within-domain validation is used for model selection and threshold calibration. The MCFD held-out split is interpreted as a camera/view-shift test. GMDCSA24 contributes a subject-disjoint held-out subject test. CAUCAFall is treated as an external cross-dataset test and is never used for tuning.

The notebook also creates a persistent artifact hierarchy under Google Drive. The source defines separate locations for ZIP archives, pose caches, metadata, results, checkpoints, and logs. This is important because the expensive portion of the workflow—video decoding plus pose inference—can be resumed from cached per-segment NPZ files instead of being repeated after an interrupted session.

Stage	Primary responsibility
Environment setup	Import scientific libraries, selectively repair missing Ultralytics, establish seeds, select CUDA/CPU device, and enforce an operational memory limit.
Persistent storage	Mount Google Drive and create directories for archives, pose features, metadata, results, models, and logs.
Runtime control	Track cumulative wall-clock budget, distinguish essential from optional stages, log budget decisions, and checkpoint elapsed time.
Dataset acquisition	Resolve versioned Zenodo files, verify expected checksums and sizes, resume partial downloads, and reuse verified caches.
Manifest construction	Convert heterogeneous dataset organization into a common segment-level schema with provenance and explicit split assignments.
Pose extraction	Sample exactly 32 temporal points, infer frozen 17-keypoint poses, select a primary person, normalize skeleton coordinates, derive motion features, and cache the result.
Feature standardization	Estimate statistics from training data only and apply the same transformation to later splits.
Modeling	Train a graph-temporal proposal plus tiny MLP and GRU alternatives and optional feature/structural ablations.
Model selection	Choose the final checkpoint using validation-only calibrated accuracy with deterministic tie-breaking.
Held-out evaluation	Evaluate MCFD, GMDCSA24, and CAUCAFall only after the model is frozen.
Diagnostics and packaging	Analyze hard negatives, record environment metadata, consolidate result files, and verify the final bundle.

Table 1: Conceptual decomposition of the notebook.

3. Technical Foundations

3.1 Binary temporal fall detection

The prediction target is binary. Each annotated segment is mapped from a source activity class to

$$y = \begin{cases} 1, & \text{fall or fallen,} \\ 0, & \text{all other annotated activities.} \end{cases} \quad (2)$$

The notebook’s class mapping identifies source labels 1 (fall) and 2 (fallen) as positive. Other classes include walking, sitting, lying, standing, kneeling, squatting, crawling, jumping, and an “other” category. This construction is scientifically useful because several non-fall actions are difficult negatives rather than trivial background clips.

3.2 Pose-based representation

For a sequence of $T = 32$ frames and $V = 17$ COCO keypoints, the raw spatial representation has coordinates

$$\mathbf{X} \in \mathbb{R}^{T \times V \times 2}. \quad (3)$$

A keypoint detector also provides a confidence value for each joint. The notebook expands the per-joint feature vector to eight channels:

$$\mathbf{f}_{t,v} = [x_{t,v}, y_{t,v}, c_{t,v}, v_{t,v}^x, v_{t,v}^y, a_{t,v}^x, a_{t,v}^y, s_{t,v}], \quad (4)$$

where c is pose confidence, (v^x, v^y) are velocity components, (a^x, a^y) are acceleration components, and s is speed magnitude.

Thus the complete cached tensor has shape

$$\mathbf{F} \in \mathbb{R}^{32 \times 17 \times 8}. \quad (5)$$

3.3 Graph representation

The 17 body joints are treated as graph nodes. The notebook defines a fixed skeleton edge set corresponding to the standard COCO body connectivity. With adjacency matrix $\mathbf{A}$, a graph-convolutional operation aggregates information over connected joints before applying a learned pointwise projection.

The implemented normalization is based on degree normalization:

$$\tilde{\mathbf{A}} = \mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}}, \quad (6)$$

with a small numerical floor in the code when degrees are near zero. This is a standard way to prevent high-degree nodes from dominating simple adjacency aggregation.

3.4 Temporal convolution

The temporal branch first flattens all joint features at each frame and projects the resulting 17C-dimensional vector to a hidden representation. It then applies one-dimensional convolutions over time, including a dilated convolution. The general form of a temporal convolution can be written as

$$\mathbf{h}_t = \sum_{k \in \mathcal{K}} \mathbf{W}_k \mathbf{x}_{t-dk}, \quad (7)$$

where d is the dilation factor and $\mathcal{K}$ is the kernel index set. The use of dilation enlarges the temporal receptive field without proportionally increasing kernel width.

3.5 Temporal self-attention

The attention block uses two attention heads through PyTorch’s `MultiheadAttention`. Given a sequence $\mathbf{H}$, self-attention computes queries, keys, and values from the same sequence:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V}. \quad (8)$$

The implementation then applies a residual connection and layer normalization,

$$\mathbf{H}' = \text{LayerNorm}(\mathbf{H} + \text{Attention}(\mathbf{H}, \mathbf{H}, \mathbf{H})). \quad (9)$$

The attention mechanism is intended to let the classifier emphasize temporally informative portions of a short movement sequence.

4. Data Sources and Input Processing

4.1 Dataset policy

The default experiment uses three datasets with different scientific roles.

Dataset	Role	Implementation policy
MCFD	Training and cross-view test	Built from OmniFall staged labels plus a published camera-disjoint CV split; interpreted strictly as view shift rather than subject-independent generalization.
GMDCSA24 v2.1	Training, validation, and held-out subject test	Subject-aware split with Subjects 1–2 for training, Subject 3 for validation, and Subject 4 for held-out testing.
CAUCAFall v5	External test only	Kept out of model development and evaluated only after the final model is frozen.

Table 2: Scientific role of each dataset in the notebook’s default experiment.

The source explicitly excludes EDF, OCCU, and UP-Fall from default acquisition. OOPS-1k remains optional. This is an example of experimental scope being constrained deliberately rather than by accident.

4.2 Versioning and provenance

The notebook resolves dataset metadata at runtime through Zenodo records and compares the remote checksum with a hard-coded expected checksum. The reported values include:

- MCFD repack: Zenodo record 17170592.
Expected MD5: b365b5a4b16691fce3fc09ccd554df79.
- GMDCSA24 v2.1: Zenodo record 13354453.
Expected MD5: 3d36f2c5c1a666b99639e4e9fd843efb.
- CAUCAFall repack: Zenodo record 17170592.
Expected MD5: 5a1899fe0022c9c4c0d37ce64ac0c986.

The report treats these as implementation metadata recorded by the notebook, not as independently re-verified external facts. The notebook itself distinguishes original scientific sources from convenience repacks and explicitly warns that a repack does not create a new license grant.

The executed notebook output recorded archive sizes of approximately 2.61 GiB for MCFD, 1.03 GiB for GMDCSA24, and 165.47 MiB for the CAUCAFall repack. These measurements describe the observed cache state, not universal dataset size claims.

4.3 Secure acquisition and extraction

The download helper writes to a `.part` file, tracks progress in bytes, can resume interrupted transfers, and rejects a remote file when the reported checksum differs from the configured expectation. A completed local archive is reused only if its checksum matches.

ZIP extraction implements a Zip Slip defense. For each archive member, the resolved target path is compared with the resolved extraction root. Any member whose canonical path escapes that root causes the extraction stage to fail. This is a meaningful security property for automated dataset handling.

4.4 Manifest as the scientific boundary

The manifest is the core contract between raw data and learning. A common row contains the dataset, physical video path, subject, camera, scenario, activity name, source label, binary label, and temporal interval. By preserving the interval before frame sampling, the implementation ensures that temporal windows are derived from the annotation rather than independently inventing a segmentation scheme.

The notebook emphasizes that split assignment is performed before segment-level sampling. Consequently, the split is attached to the manifest row itself and propagated into pose extraction rather than recomputed after frames have been generated.

The observed executed state recorded 1,352 MCFD manifest rows, 458 GMDCSA24 rows, and 258 CAUCAFall rows. The corresponding binary label distributions were 440/912 positive/non-positive for MCFD, 155/303 for GMDCSA24, and 98/160 for CAUCAFall. These values describe the manifest construction stage contained in the notebook output; they are not model-performance results.

5. Split Design and Leakage Control

The notebook takes a conservative position on leakage.

For GMDCSA24, subjects are explicitly assigned to train, validation, and test groups. This makes the held-out test set subject-disjoint from training.

For MCFD, the split comes from published cross-view metadata. Because synchronized cameras can observe the same physical event from different views, the notebook explicitly avoids calling this a subject-independent or event-independent generalization benchmark. The scientifically relevant interpretation is camera/view shift.

CAUCAFall is external-only. It is not used to select the model, tune the threshold, or create the training representation statistics. This is the strongest form of held-out protection implemented in the notebook.

The loader also never performs frame-level random splitting. Instead, each complete manifest segment is an example. This is important because randomly assigning frames from the same video to different splits would create severe temporal leakage.

6. Runtime and Resource Management

6.1 Three-hour target and hard ceiling

The notebook separates a target budget from an absolute ceiling. The configured target is 3 hours, while the runtime state defaults to 3.83 hours for its hard operational limit. Optional stages can be skipped after the target is exceeded, whereas essential stages are allowed to continue until the hard ceiling.

Let t_c denote cumulative completed runtime and t_s the elapsed time of the current session. The reported total is conceptually

$$t_{\text{total}} = t_c + t_s. \quad (10)$$

The runtime controller persists this state to JSON and records budget decisions line-by-line in a JSONL log.

A completed run archives the previous budget state and resets the cumulative timer while preserving datasets, pose caches, and checkpoints. This is an efficient separation of reusable computational artifacts from per-run wall-clock accounting.

6.2 Pose throughput estimation

Before large-scale pose extraction, the notebook inspects a sample of videos and benchmarks real YOLO11n-Pose inference on actual decoded frames. The executed notebook output reported a median of approximately 236 frames/s using batch size 32, image size 640, and FP16 on a Tesla T4.

The source also estimated 22,496 sampled train/validation frames from 703 segments and reported a conservative pose-extraction estimate of about 3.9 minutes. This is an engineering estimate rather than a scientific result.

6.3 VRAM control

The operational memory threshold is 13.5 GiB of allocated peak memory. After expensive stages, the code calls `torch.cuda.max_memory_allocated()` and aborts if the measured peak exceeds the configured threshold.

This does not guarantee that a future environment will behave identically, but it creates a reproducible failure condition instead of allowing memory consumption to grow silently.

7. Pose Extraction Pipeline

7.1 Temporal sampling

The sampler converts the annotation interval $[t_s, t_e]$ into frame indices using the video frame rate f :

$$i_s = \text{round}(t_s f), \quad i_e = \text{round}(t_e f). \quad (11)$$

It then chooses $T = 32$ approximately uniformly spaced indices using

$$\left\{ \text{round} \left(i_s + \frac{k}{T-1} (i_e - i_s) \right) \right\}_{k=0}^{T-1}. \quad (12)$$

OpenCV is used for decoding. The canonical reader seeks to the start frame and reads forward through the requested interval, preserving only the target frames. A benchmark-specific reader contains a sequential fallback for codecs where random seeking is unreliable.

7.2 Primary-person selection

YOLO11n-Pose may detect multiple people. The notebook therefore chooses one person per frame. The score for candidate i is

$$S_i = 0.45 b_i + 0.30 k_i + 0.15 A'_i + 0.10 C_i, \quad (13)$$

where b_i is bounding-box confidence, k_i is mean keypoint confidence, A'_i is a clipped normalized area term, and C_i is a continuity score relative to the previous selected center.

The continuity term is based on normalized Euclidean displacement:

$$C_i = \max \left(0, 1 - \frac{\|\mathbf{c}_i - \mathbf{c}_{t-1}\|_2}{\sqrt{H^2 + W^2}} \right). \quad (14)$$

This is a heuristic identity-tracking mechanism, not a formal multi-object tracker. Its role is to reduce abrupt subject switching in scenes with multiple detections.

7.3 Missing detections

When no suitable person is detected, the implementation inserts a 17-by-2 array of NaNs and zero confidence values. Those missing coordinates are later replaced by zeros after normalization and feature construction.

This is an important semantic point. A zero coordinate in the final tensor does not necessarily mean a true body position at the origin; it can represent missing or invalid evidence after the pipeline has converted unavailable observations into numeric values.

7.4 Skeleton normalization

The skeleton is translated and scaled frame-by-frame.

The center is selected from the mean of the hips when available, otherwise the shoulders, otherwise all finite joints. Let this center be $\mathbf{m}_t$.

The scale is the maximum of shoulder width, torso length, and 1:

$$s_t = \max \left(\|\mathbf{p}_{\text{shoulder}}^{(L)} - \mathbf{p}_{\text{shoulder}}^{(R)}\|_2, \|\mathbf{m}_{\text{hip}} - \mathbf{m}_{\text{shoulder}}\|_2, 1 \right). \quad (15)$$

For every valid joint,

$$\tilde{\mathbf{p}}_{t,v} = \frac{\mathbf{p}_{t,v} - \mathbf{m}_t}{\max(s_t, 10^{-3})}. \quad (16)$$

Missing centers and scales are interpolated across time where possible. This design makes the representation less dependent on absolute image position and person size.

7.5 Velocity, acceleration, and speed

The code uses the sampled timestamps to approximate first and second derivatives. Let Δt_t denote the local temporal spacing. Velocity is

$$\mathbf{v}_t \approx \frac{\partial \tilde{\mathbf{p}}_t}{\partial t}, \quad (17)$$

implemented through `np.gradient`. Acceleration is then

$$\mathbf{a}_t \approx \frac{\partial \mathbf{v}_t}{\partial t}. \quad (18)$$

Speed is the Euclidean norm

$$s_t = \|\mathbf{v}_t\|_2. \quad (19)$$

The final feature tensor is concatenated in the order

$$[x, y, c, v_x, v_y, a_x, a_y, s]. \quad (20)$$

NaNs and infinite values are replaced with zero before the feature array is stored as `float32`.

7.6 Pose cache integrity

Each segment is cached as a compressed NPZ file. Cache reuse requires all of the following:

1. feature shape exactly (32, 17, 8);
2. stored binary label equal to the manifest label;
3. stored cache version equal to the current cache version.

The cache version records the extraction configuration, including YOLO11n-Pose, sequence length, channel count, batch size, and FP16 policy. Extraction writes to a temporary NPZ file and then uses `os.replace`, reducing the risk of leaving a partially written final cache.

The executed notebook state reported 703 successful train/validation pose rows, of which 365

were cache hits and 338 were freshly extracted. No failed rows remained at that stage.

8. Train-Only Standardization

The notebook fits feature statistics using training examples only. This prevents validation, test, or external distributions from influencing the normalization parameters.

For channel j , the code computes

$$\mu_j = \frac{1}{N} \sum_{n=1}^N x_{n,j}, \quad (21)$$

and

$$\sigma_j = \sqrt{\max\left(\frac{1}{N} \sum_{n=1}^N x_{n,j}^2 - \mu_j^2, 0\right)}. \quad (22)$$

The standardizer is then applied as

$$z_{n,j} = \frac{x_{n,j} - \mu_j}{\max(\sigma_j, 10^{-4})}. \quad (23)$$

Confidence is deliberately excluded from z-scoring so that the confidence channel retains its native interpretation in $[0, 1]$.

The executed notebook recorded the fitted statistics. Those statistics show that the motion derivatives have much larger raw numeric scales than normalized coordinates, justifying the train-only standardization step.

9. Dataset Class and Augmentation

9.1 Tensor preparation

The `PoseSequenceDataset` class loads each NPZ file on demand, applies train-derived standardization to channels 0, 1, 3, 4, 5, 6, and 7, and then applies an optional binary channel mask.

The mask therefore acts multiplicatively:

$$\mathbf{X}'_{t,v,c} = m_c \mathbf{X}_{t,v,c}, \quad m_c \in \{0, 1\}. \quad (24)$$

The default full proposed mask is

$$\mathbf{m} = [1, 1, 0, 1, 1, 1, 1, 0]. \quad (25)$$

Thus the default proposed experiment actually uses position, velocity, and acceleration while suppressing confidence and speed.

9.2 Skeleton-only augmentation

The training dataset optionally applies two inexpensive augmentations.

First, with probability 0.5, Gaussian noise with standard deviation 0.01 is added to normalized coordinates:

$$\tilde{\mathbf{p}} \leftarrow \mathbf{p} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, 0.01^2). \quad (26)$$

Second, with probability 0.25, the temporal order is reversed.

The label remains unchanged. The reversal operation implicitly assumes that the distinction between the positive and negative classes should remain valid under time reversal. This may be useful as a cheap robustness augmentation, but it also removes directionality information from those augmented examples. Whether that assumption is appropriate is an empirical question not resolved by the notebook.

10. Model Architecture

10.1 Graph-convolutional spatial branch

The graph branch expects a tensor arranged as $B \times C \times T \times V$. It performs adjacency aggregation through

$$\mathbf{Z}_{n,c,t,w} = \sum_v \mathbf{X}_{n,c,t,v} \tilde{A}_{v,w}, \quad (27)$$

implemented using `torch.einsum`. A 1×1 convolution then mixes channels, followed by batch normalization and ReLU.

Two graph-convolution blocks are interleaved with temporal convolutions of kernel sizes 5 and 3. The output is averaged over joints, giving a sequence

$$\mathbf{S} \in \mathbb{R}^{B \times T \times H}, \quad H = 48. \quad (28)$$

10.2 Temporal branch

The temporal branch flattens the 17 joints and C channels at each time point and applies a learned linear projection:

$$\mathbf{u}_t = \mathbf{W} \text{vec}(\mathbf{X}_t) + \mathbf{b}, \quad \mathbf{u}_t \in \mathbb{R}^{48}. \quad (29)$$

This sequence is passed through two temporal convolutions. The second is dilated by a factor of 2.

The branch produces

$$\mathbf{T} \in \mathbb{R}^{B \times T \times 48}. \quad (30)$$

10.3 Gated fusion

The two branch outputs are fused by a learned gate. The code computes frame-level confidence statistics first:

$$\mathbf{q}_t = \begin{bmatrix} \text{mean}_v(c_{t,v}) \\ \text{std}_v(c_{t,v}) \end{bmatrix}. \quad (31)$$

These statistics are concatenated with $\mathbf{S}_t$ and $\mathbf{T}_t$ and passed through a two-layer sigmoid gate:

$$\mathbf{g}_t = \sigma(\mathbf{W}_2 \text{ GELU}(\mathbf{W}_1[\mathbf{S}_t; \mathbf{T}_t; \mathbf{q}_t] + \mathbf{b}_1) + \mathbf{b}_2). \quad (32)$$

The fused representation is

$$\mathbf{F}_t = \mathbf{g}_t \odot \mathbf{S}_t + (1 - \mathbf{g}_t) \odot \mathbf{T}_t. \quad (33)$$

This is the conceptual center of the proposed classifier: spatial structure and temporal dynamics are not simply concatenated; the network learns how much to rely on each stream at each time point.

10.4 Attention and classification

When enabled, the two-head attention block refines the fused sequence. The sequence is then mean-pooled over time:

$$\bar{\mathbf{F}} = \frac{1}{T} \sum_{t=1}^T \mathbf{F}_t. \quad (34)$$

The final head consists of layer normalization, dropout, and a linear scalar output. The scalar is a logit z and is converted to a fall probability through the logistic function

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}. \quad (35)$$

The training target is binary, so the natural loss used by the implementation is binary cross-entropy with logits.

10.5 Baselines

The tiny MLP flattens all $32 \times 17 \times 8$ features and maps them through a 64-unit hidden layer.

The tiny GRU first projects each frame’s 17×8 features to 48 dimensions, processes the sequence with a GRU, and uses the final hidden state for classification.

These baselines are intentionally small. Their value is methodological: they establish whether the more structured graph-temporal architecture provides evidence beyond a simple flattened classifier or a conventional recurrent sequence model.

11. Training Procedure

The default training configuration is:

Parameter	Value	Role
Sequence length	32	Fixed temporal input size
Batch size	64	Training batch size
Epochs	18	Maximum training epochs
Patience	4	Early stopping threshold
Learning rate	3×10^{-4}	AdamW step size
Weight decay	10^{-4}	AdamW regularization
Dropout	0.2	Regularization in model/data path
Gradient clipping	1.0	Maximum gradient norm
Workers	2	DataLoader workers
Weighted sampler	disabled	Avoid combined imbalance mechanisms
Positive loss weighting	disabled	Same rationale as above

Table 3: Training configuration encoded in the notebook.

The optimizer is AdamW. Gradients are optionally computed under automatic mixed precision on CUDA, and the implementation uses a gradient scaler. Each training batch follows the standard sequence

$$\theta \rightarrow \text{forward} \rightarrow \mathcal{L} \rightarrow \nabla_{\theta} \mathcal{L} \rightarrow \text{clip} \rightarrow \text{AdamW update.} \quad (36)$$

The code clips gradients using `clip_grad_norm_` before the optimizer step.

The training loss is computed with `BCEWithLogitsLoss`. The source supports an optional positive class weight, but the default configuration explicitly turns it off because the observed class imbalance is judged moderate and the weighted sampler is also disabled.

Each improved validation checkpoint stores the model state, optimizer state, epoch, validation metrics, threshold, history, experiment signature, and configuration. The signature includes the model name, data identities, split information, and channel mask. A checkpoint is therefore not reused blindly when the experiment definition changes.

12. Validation and Threshold Calibration

12.1 Metric computation

The notebook computes accuracy, precision, recall/sensitivity, F1, specificity, false-positive rate, false-negative rate, ROC-AUC, PR-AUC, and the confusion-matrix counts.

For predicted labels $\hat{y}$,

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (37)$$

Precision and recall are

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (38)$$

F1 is

$$F_1 = \frac{2 \text{ Precision Recall}}{\text{Precision} + \text{Recall}}. \quad (39)$$

The false-positive rate is

$$\text{FPR} = \frac{FP}{FP + TN}, \quad (40)$$

and the false-negative rate is

$$\text{FNR} = \frac{FN}{FN + TP}. \quad (41)$$

ROC-AUC and average precision are computed from the continuous probabilities when both classes are present.

12.2 Validation-only operating threshold

Instead of always using 0.5, the notebook searches a coarse threshold grid from 0.20 to 0.80 in increments of 0.025.

For threshold τ ,

$$\hat{y}_i(\tau) = \mathbb{I}[p_i \geq \tau]. \quad (42)$$

The selected threshold maximizes validation accuracy, then F1, then minimizes false-positive rate, and finally prefers the value nearest to 0.5.

This calibration is performed independently of held-out test data. During training, the validation threshold is used only to evaluate the current validation operating point. Test data are explicitly deferred until all models have been trained and the final model has been selected.

13. Model Suite and Selection Rule

The main suite contains the full proposed model, two small baselines, three channel ablations, and two structural ablations when runtime permits.

The selection rule is deterministic:

1. identify the maximum validation calibrated accuracy;
2. keep all models within 0.002 accuracy points of that maximum;
3. within that candidate set, prefer higher validation F1;
4. then prefer lower validation false-positive rate;
5. then prefer fewer parameters.

This is implemented without touching test data.

The source labels the full proposed model as essential and the additional baselines/ablations as optional. Thus, the computational budget is treated as part of experimental design: the primary scientific comparison is protected even if optional comparisons have to be skipped.

14. Final Model Freezing

Once the selection rule produces `chosen_name`, the notebook reconstructs the corresponding model family exactly, maps the selected channel mask, loads the associated checkpoint, and performs a strict state-dictionary compatibility check.

The checkpoint loader explicitly uses `weights_only=False` for newer PyTorch versions because the notebook checkpoint contains metadata and optimizer state in addition to the model state dictionary. The code then loads only the `model` state dictionary into the freshly recreated architecture.

The final model is switched to evaluation mode and the operating threshold is recalibrated once more using validation data only. This produces `final_threshold`, which is subsequently used for held-out evaluation.

Conceptually, the test protocol is therefore

$$\begin{aligned} &\text{train} \rightarrow \text{validate and select} \rightarrow \text{freeze model} \\ &\quad \rightarrow \text{calibrate threshold on validation} \rightarrow \text{evaluate test.} \end{aligned} \tag{43}$$

15. Held-Out Testing

15.1 GMDCSA24 subject-disjoint test

The GMDCSA24 held-out set consists of the rows assigned to `test`, which corresponds to Subject 4 under the notebook’s fixed split. It is evaluated with the final feature mask and frozen model without augmentation.

This is the notebook’s strongest within-dataset identity-disjoint evaluation.

15.2 MCFD cross-view evaluation

MCFD test rows are drawn from the versioned camera-disjoint split. The notebook explicitly records the interpretation as `view_shift_not_subject_independent`.

This distinction is scientifically important. Because multiple cameras can observe synchronized instances of the same scenario, performance across MCFD views is informative about viewpoint robustness but cannot establish strong subject-independent generalization.

15.3 CAUCAFall external evaluation

CAUCAFall pose extraction occurs only after the final model is frozen. A cached pose manifest is reused only when the manifest length, required columns, and referenced cache paths are valid.

The final external evaluation produces the same family of binary metrics as the within-domain evaluation. The external interpretation is explicitly recorded as an unseen-dataset environment.

The notebook therefore has three distinct generalization questions:

16. Targeted Failure Recovery

One technically notable part of the notebook is its handling of held-out MCFD extraction failures.

The canonical extractor is attempted first. If a physical MCFD clip is shorter than the absolute annotation end time, the notebook treats that physical file as a pre-segmented clip and retries

Evaluation	Source of shift	What it can support
GMDCSA24 validation	unseen subject during model development	validation-based model selection and threshold calibration.
GMDCSA24 test	unseen subject within dataset	subject-disjoint within-dataset evaluation.
MCFD test	unseen camera/view	view-shift robustness, not subject-independent generalization.
CAUCAFall test	unseen dataset/environment	external cross-dataset evaluation.

Table 4: Interpretation of the notebook’s evaluation boundaries.

extraction over the full physical clip rather than the original absolute interval. Batch size is also reduced through a sequence such as 32, 16, 8, 4 when retries are necessary.

This logic is deliberately targeted: labels, split assignments, and the final model are not modified. Successful rows are retained unchanged, and only unresolved failures are retried.

The scientific trade-off should nevertheless be recognized. The physical-clip fallback changes the temporal sampling semantics for affected rows from “annotation interval within a longer stream” to “entire already-segmented clip”. The notebook documents this behavior rather than hiding it. It is a pragmatic engineering repair, not evidence that the two interpretations are theoretically identical.

17. Hard-Negative Analysis

The hard-negative stage focuses on non-fall activities that are plausible false alarms. The taxonomy includes walking, sitting, lying, standing, kneeling, squatting, crawling, jumping, and other annotated non-fall categories.

For a hard-negative activity a , the notebook aggregates

$$\text{FAR}(a) = \frac{\text{FalseAlarms}(a)}{N(a)}, \quad (44)$$

along with the mean predicted fall probability.

This is scientifically useful because aggregate accuracy can hide which benign activities are most easily mistaken for falls.

There is, however, an implementation inconsistency: the hard-negative table recomputes the binary prediction using the fixed threshold 0.5, even though the preceding evaluation call uses `final_threshold`. Consequently, the false-alarm counts and rates in the hard-negative table are not guaranteed to correspond to the actual final operating point. A corrected implementation should pass `final_threshold` into the aggregation function or otherwise compute false alarms from the already-thresholded predictions.

18. Reproducibility and Reliability

The notebook records a large amount of environment information:

- random seed;
- measured runtime and runtime limits;

- GPU memory budget;
- dataset configuration;
- model architecture settings;
- training configuration;
- split protocol;
- example counts;
- Python, PyTorch, CUDA, Ultralytics, and OpenCV versions;
- GPU name and peak allocated/reserved memory;
- chosen model and selection rule;
- final validation-calibrated threshold.

The executed environment output in the notebook recorded Python 3.13.15, NumPy 2.1.3, pandas 2.2.3, OpenCV 5.0.0, PyTorch 2.11.0+cu128, Ultralytics 8.3.176, and a Tesla T4 GPU. These are observations from the captured notebook execution state.

The final bundling stage copies results, metadata, models, and logs into a ZIP file and verifies that required output files exist. The captured execution state reports that the final verification found all required outputs and both optional outputs, with a cumulative measured runtime of approximately 29.92 minutes.

Those metadata establish that the engineering pipeline itself was executed successfully to the packaging stage. They do not establish that every model experiment described in the source was completed with valid performance results; the training/evaluation cells do not contain persisted numeric output in the uploaded notebook. Accordingly, this report makes no model-performance claim.

19. Implementation Deviations and Important Caveats

19.1 The confidence-aware claim is only partially realized

The notebook describes the architecture as confidence-aware. The model code indeed computes confidence mean and standard deviation and provides them as inputs to the fusion gate.

However, the default proposed feature mask is

$$[1, 1, 0, 1, 1, 1, 1, 0], \quad (45)$$

whose third element disables the confidence channel. Because the gate reads the confidence channel directly from the masked input tensor,

$$c = x[:, :, :, 2], \quad (46)$$

the confidence statistics are identically zero in the default full proposed experiment. Therefore the confidence-conditioned fusion mechanism does not receive meaningful confidence information under that default mask.

This is not a cosmetic discrepancy. It changes the effective architecture from “confidence-aware fusion” to fusion without informative confidence input. A future corrected experiment should

either include confidence in the full mask or remove the confidence-gating claim from the description.

19.2 The spatial branch contains unused confidence code

Inside `STGCNBranch.forward`, the source creates a tensor named `conf` filled with zeros and comments that confidence should be used for confidence-weighted node pooling. That tensor is not used in the returned expression. The actual pooling is a simple mean across joints:

$$\mathbf{S}_t = \frac{1}{V} \sum_{v=1}^V \mathbf{H}_{t,v}. \quad (47)$$

Thus, confidence-weighted node pooling is documented in comments but is not implemented in the returned computation.

19.3 Hard-negative threshold mismatch

As noted earlier, hard-negative false-alarm rates are based on threshold 0.5 even when the final model uses a validation-calibrated threshold different from 0.5. This can make the hard-negative report inconsistent with the final operating point.

19.4 Per-frame normalization

Skeleton translation and scaling are performed independently at each frame. This removes some information about global motion and absolute scale. That is a deliberate invariance, but it may also suppress cues such as whole-body displacement through the scene.

The notebook partially recovers temporal information through velocity and acceleration of the normalized joints, but the resulting representation remains fundamentally relative to the body-centered coordinate system.

19.5 Temporal reversal augmentation

The reversal augmentation is computationally cheap, but reversing a sequence assumes that the target label is unchanged and that reversed temporal dynamics are useful training variations. For a fall detector, the temporal direction can matter because a fall often contains a characteristic transition from upright to ground contact. The code does not test this assumption separately.

19.6 Heuristic person selection

The primary-person score is a fixed weighted heuristic. It is not learned and does not maintain an explicit identity state beyond the previous center. In crowded scenes, occlusions or crossings could therefore cause identity switches.

19.7 View-shift versus independent-event generalization

The notebook’s treatment of MCFD is appropriately cautious, but the limitation is structural. A camera-disjoint split can still reuse the underlying event across views. Consequently, MCFD should not be used as the sole basis for claiming robust generalization to unseen people or unseen events.

20. Computational Complexity

Let T be the sequence length, V the number of joints, and C the number of channels.

The graph aggregation step is approximately

$$O(BCTV^2) \quad (48)$$

for a dense adjacency multiplication, although a sparse implementation could reduce this when the graph is sparse.

The temporal branch's input projection is approximately

$$O(BTVCH), \quad (49)$$

where H is the hidden width.

The one-dimensional temporal convolutions scale approximately with

$$O(BTH^2K), \quad (50)$$

for hidden width H and kernel width K .

For temporal self-attention, the dominant interaction term is quadratic in sequence length:

$$O(BT^2H). \quad (51)$$

Because $T = 32$ and $H = 48$, this term is small compared with attention over long video sequences. This is consistent with the notebook's objective of achieving useful temporal modeling without introducing a large Transformer.

Memory consumption during training scales with batch size and intermediate activations. The main practical memory pressure comes earlier, during image-based pose extraction, which is why the notebook combines batched FP16 inference with explicit VRAM monitoring.

21. Code Organization and Execution Semantics

The notebook is organized into logical stages, but it also contains defensive repairs and retry layers that are more detailed than the conceptual pipeline. The execution can be divided into five broad software layers.

First is an environment layer: dependency inspection, imports, device setup, reproducibility seeds, and memory checks.

Second is a data layer: downloads, checksums, extraction, manifest construction, split assignment, and metadata persistence.

Third is a representation layer: frame sampling, pose estimation, subject selection, skeleton normalization, derivative features, caching, and standardization.

Fourth is a learning layer: datasets, model classes, training utilities, checkpoints, validation, model comparison, selection, and final freezing.

Fifth is an evaluation and artifact layer: held-out pose extraction, cross-view testing, external testing, hard-negative analysis, reproducibility metadata, plots generated by the original notebook, result packaging, and final verification.

The notebook is therefore better understood as an executable research pipeline than as a single machine-learning script.

22. Overall Technical Assessment

The strongest aspect of the implementation is its experimental discipline. The source makes unusually explicit distinctions between raw data provenance, train/validation/test boundaries, external testing, resource budgets, cached intermediate representations, and final artifact verification.

The frozen-pose strategy is also technically coherent for a compute-limited setting. The pose estimator is isolated from the classifier, and the trainable model works on a compact structured sequence rather than raw frames. Position, velocity, and acceleration provide complementary information: position encodes posture, velocity describes movement, and acceleration captures changes in movement dynamics.

The model architecture is appropriately lightweight. The ST-GCN branch encodes body topology, the TCN branch emphasizes temporal dynamics, the gated fusion combines complementary representations, and temporal attention can reweight informative frames. The tiny MLP and GRU provide meaningful low-cost reference points.

The notebook’s scientific caution around MCFD is particularly good. Explicitly refusing to turn a cross-view split into a subject-independent claim shows awareness of what the data can and cannot establish.

The principal weakness is not the overall architecture but the presence of implementation and documentation mismatches. The confidence-aware claim is not supported by the default feature mask; confidence-weighted spatial pooling is described in a comment but not executed; and hard-negative false alarms use a different threshold from final evaluation. These details should be corrected before the notebook is presented as a finalized research artifact.

A second limitation is evidentiary rather than algorithmic: the uploaded notebook contains completed preprocessing and packaging outputs, but it does not preserve completed model-training metrics in its captured outputs. As a result, this report can explain the methodology rigorously but cannot responsibly claim that the proposed model outperformed the baselines or achieved any particular accuracy, F1, ROC-AUC, PR-AUC, or external generalization level.

23. Conclusion

The notebook implements a compact, reproducible fall-detection pipeline centered on frozen human pose estimation and temporal skeleton modeling. Its computational path begins with versioned video archives and a provenance-aware manifest, continues through 32-frame pose extraction and body-centered normalization, augments the representation with velocity and acceleration, standardizes trainable channels using training data only, and feeds the resulting sequence into a lightweight combination of graph convolution, temporal convolution, gated fusion, and temporal attention.

Equally important, the notebook treats experimental control as a first-class engineering concern. It protects against data leakage, records dataset and environment metadata, caches expensive pose computations, monitors runtime and GPU memory, supports checkpoint resumption, and

verifies final artifacts before marking a run complete.

From a research-portfolio perspective, the implementation demonstrates several useful competencies at once: structured data engineering, computer-vision preprocessing, graph and temporal modeling, PyTorch training, evaluation design, reproducibility engineering, and scientific caution about benchmark interpretation.

The main next step for scientific maturity is not to make the model larger. It is to align the implementation precisely with its own stated claims: either restore confidence as an active default feature, remove unused confidence-aware components, make hard-negative thresholds consistent with final evaluation, and preserve completed training/evaluation outputs alongside the notebook. Once those details are reconciled, the pipeline will have a cleaner correspondence between its scientific narrative and its executable behavior.

Traceability Notes

The following map identifies the principal notebook regions used for this report.

Report topic	Notebook cells	Primary implementation evidence
Environment and reproducibility	3–6	Dependency policy, seed initialization, device selection, VRAM budget, persistent runtime state.
Dataset provenance and download	7–11	Dataset configuration, Zenodo resolution, checksum validation, resumable download, safe extraction.
Manifest construction and splits	12–15	OmniFall labels, dataset-specific parsing, split assignment, leakage guard.
Runtime estimation	16–18	Video statistics and real YOLO11n-Pose throughput benchmark.
Pose representation	19–21	Sampling, primary-person selection, normalization, motion features, caching, retry logic.
Standardization	22–23	Train-only mean/std estimation and persistence.
Models	24–25	Pose dataset, graph convolution, ST-GCN, TCN, attention, proposal model, MLP, GRU.
Training and metrics	26–33	DataLoaders, loss, optimizer, calibration, checkpointing, model selection, final freezing.
Held-out testing	34–37	Test-pose extraction, MCFD view-shift evaluation, GMDCSA24 held-out subject evaluation, CAUCAFall external test.
Hard negatives and packaging	38–47	Activity-specific false alarms, metadata, result bundle, guardrails, final verification.